

PhotonFreak Backend Status

Current Status

Backend Server Running Successfully

API Server: <http://localhost:8000>

API Documentation: <http://localhost:8000/docs>

Architecture

The unified FastAPI backend integrates three subsystems:

1. **Fault Detection** - Image-based solar panel fault prediction
2. **Charging Optimization** - Battery charging schedule optimization
3. **Solar Forecasting** - Probabilistic solar irradiance forecasting
4. **Battery Health** (Optional) - Battery state of health estimation

Running the Backend

Start Server

```
cd backend
pip install -r requirements.txt
uvicorn app.main:app --port 8000
```

Without Reload (Production)

```
cd backend
uvicorn app.main:app --port 8000
```

Health Status

Endpoint

```
curl http://localhost:8000/api/health
```

Response (Python 3.13)

```
{
  "ok": true,
```

```
"fault_model_ready": true,  
"solar_models_ready": true,  
"battery_model_ready": false,  
"battery_module_available": false  
}
```

API Endpoints

Health Checks

- **GET /api/health** - Overall API status

Fault Prediction

- **POST /api/faults/predict** - Predict panel faults from image
 - Input: JPG/PNG/WebP image file
 - Output: Fault prediction with confidence score

Charging Optimization

- **POST /api/charging/optimise** - Optimize battery charging schedule
 - Input: Battery specs, solar forecast, grid prices
 - Output: Hour-by-hour charging schedule

Solar Forecasting

- **POST /api/solar/bootstrap** - Load historical weather/production data
 - Input: CSV with historical hourly readings
 - Output: Success confirmation
- **POST /api/solar/readings** - Submit current reading, get forecast
 - Input: Current meteorological conditions
 - Output: Probabilistic forecast (q05, q50, q95) with EV charging signal
- **GET /api/solar/history** - Retrieve stored readings
 - Query: hours=48 (default)
 - Output: Array of readings with timestamps

Battery Health (503 Service Unavailable - TensorFlow compatibility)

- **GET /api/battery/health** - Health check
- **GET /api/battery/csv-template** - Get CSV template
- **POST /api/battery/predict** - Predict battery health
- **POST /api/battery/explain** - Generate explanation

Known Issues & Solutions

Issue: Battery Module Unavailable

Reason: Python 3.14 doesn't have TensorFlow 2.21.0 wheels yet

Status: Gracefully degraded - returns 503 Service Unavailable

Solution:

- Option A: Downgrade to Python 3.13

```
# Using conda
conda install python=3.13
# Or using pyenv
pyenv install 3.13.1
pyenv local 3.13.1
```

- Option B: Use alternative TensorFlow build (if available for arm64)

Issue: Solar Models Not Found

File: `models.pkl`

Status: Warning logged, endpoint handles gracefully

Solution: Provide `models.pkl` file from original solar forecasting project

```
# Copy from original project
cp /path/to/solar-forecasting/models.pkl backend/models.pkl
```

Issue: Optional LLM Explanations

Environment Variable: `GRQ_API_KEY`

Status: Optional - uses fallback templates if not set

Setup:

```
# Create .env in backend directory
echo "GRQ_API_KEY=your_key_here" > backend/.env
```

Environment Setup

Python Environment

Currently using Python 3.13 (conda base)

```
python --version # Python 3.13
```

Dependencies Installed

- FastAPI, Uvicorn, Pydantic (API)
- NumPy, Pandas, SciPy (Data)
- Scikit-Learn, LightGBM (ML)
- PyTorch, TorchVision (Vision)
- Python-dotenv, Requests (Utils)
- **TensorFlow**: Not installed (Python 3.14 incompatibility)

Configuration

CORS Settings

Default allowed origins (from `.env`):

```
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:5173
```

Database

SQLite file: `backend/solar_readings.db`

- Automatically created on first solar module initialization
- Stores readings, VMD cache, metadata

Next Steps

1. [OPTIONAL] Install Battery Module

- Downgrade to Python 3.13
- Run `pip install tensorflow>=2.21.0`

2. [REQUIRED] Add Solar Models

- Locate `models.pkl` from original solar project
- Copy to `backend/models.pkl`

3. [OPTIONAL] Configure Groq API

- Add `GROQ_API_KEY` to `backend/.env`
- Enables AI-powered explanation generation

4. Test Full Stack

- Start frontend: `npm run dev`
- Test all endpoints via Swagger UI
- Verify end-to-end workflows

Testing

Quick Test

```
# Health check
curl http://localhost:8000/api/health | jq

# Swagger UI
open http://localhost:8000/docs
```

Full Test (when models.pkl available)

```
# Test solar bootstrap (if you have historical CSV)
curl -X POST http://localhost:8000/api/solar/bootstrap \
  -F "file=@data.csv"

# Test fault prediction (if you have panel image)
curl -X POST http://localhost:8000/api/faults/predict \
  -F "image=@panel.jpg"
```

Debugging

Check Server Logs

```
# In terminal running uvicorn
# Watch for startup messages and any errors
```

Import Check

```
cd backend
python -c "from app.main import app; print('✓ All modules imported')"
```

Module Availability

```
cd backend
python -c "from app.main import BATTERY_AVAILABLE; print(f'Battery: {BATTERY_AVAILABLE}')"

```

File Structure

```
backend/
├── app/
│   ├── main.py                # Unified API
│   ├── model.py               # Fault prediction
│   ├── optimizer.py           # Charging optimization
│   ├── solar/                 # Solar forecasting module
│   │   ├── db.py
│   │   ├── model_service.py
│   │   ├── feature_engineering.py
│   │   ├── vmd_service.py
│   │   ├── explain_service.py
│   │   └── config.py
│   └── battery/               # Battery health module (optional)
│       ├── model_utils.py
│       ├── feature_engineering.py
│       ├── explainability.py
│       ├── schemas.py
│       └── artifacts/
├── requirements.txt           # Python dependencies
├── .env.example               # Environment template
└── solar_readings.db          # SQLite (auto-created)
```

Documentation

- Full API documentation: [INTEGRATION_GUIDE.md](#)
- Architecture overview: See main INTEGRATION_GUIDE.md in project root